

SQL Performance & Optimization Test

Scenario-based · 5 questions · ~60 minutes

You may reference syntax documentation. For each question, explain your reasoning — the approach matters as much as the final query. State any assumptions you make about the database engine.

DATASET

Brazilian E-Commerce Public Dataset by Olist

Real e-commerce data — ~100,000 orders placed across Brazilian marketplaces between 2016 and 2018. Multiple linked tables covering orders, items, payments, products, and customers.

kaggle.com/datasets/olistbr/brazilian-ecommerce

olist_orders_dataset	olist_order_items_dataset	olist_order_payments_dataset
order_id (PK)	order_id (FK)	order_id (FK)
customer_id	order_item_id	payment_sequential
order_status	product_id (FK)	payment_type
order_purchase_timestamp	seller_id	payment_installments
order_approved_at	shipping_limit_date	payment_value
order_delivered_timestamp	price	
order_estimated_delivery_date	freight_value	

olist_products_dataset	olist_order_customer_dataset
product_id (PK)	customer_id (PK)
product_category_name	customer_unique_id
product_name_length	customer_zip_code_prefix
product_description_length	customer_city
product_photos_qty	customer_state
product_weight_g	
product_length_cm	
product_height_cm	
product_width_cm	

QUESTIONS

1 The slow monthly revenue report

The finance team runs a monthly report that scans the full orders, order_items, and products tables to calculate revenue by product category. On ~100k orders it already takes over 2 minutes — and the dataset is expected to grow 10x next year.

Write a query that calculates total revenue (sum of price) and total orders per product category per month, joining olist_orders_dataset, olist_order_items_dataset, and olist_products_dataset. Only include orders with order_status = 'delivered'.

Then identify at least two structural changes you would make to bring the runtime down at current scale, and explain what breaks first when the data grows 10x.

Skills tested: multi-table JOIN · GROUP BY · filtering · indexing · partitioning

2 Spot the subquery trap

A colleague wrote the following query to find all orders whose payment value is above the average payment value for their payment type. It returns correct results but runs noticeably slower than expected.

```
SELECT o.order_id, o.customer_id, p.payment_value, p.payment_type
FROM olist_orders_dataset o
JOIN olist_order_payments_dataset p ON o.order_id = p.order_id
WHERE p.payment_value > (
  SELECT AVG(p2.payment_value)
  FROM olist_order_payments_dataset p2
  WHERE p2.payment_type = p.payment_type
)
```

Identify the performance problem. Rewrite it.

Explain what the engine is likely doing with the original version on every row, and what your rewrite does differently.

Skills tested: correlated subquery · CTE · subquery in FROM · query plan

3 Late deliveries — window function vs. self-join

The logistics team wants to flag orders where the delivery took longer than the previous order delivered to the same customer state, on the same day. A junior analyst tried a self-join on olist_orders_dataset but it timed out.

Rewrite this using a window function.

Then explain: (a) why the self-join approach caused a timeout, and (b) if this query needs to run daily on a continuously growing table, what pipeline-level change would you make so it never becomes a bottleneck?

Skills tested: LAG() · PARTITION BY · ORDER BY · self-join cost · incremental processing

4 The UNION anti-pattern

A report needs to combine 'credit_card' and 'boleto' payment orders from olist_order_payments_dataset for the year 2018, then count orders per payment type. A teammate wrote it using UNION with filters applied on the outer query. A senior engineer flags it as inefficient.

Write the anti-pattern version your teammate likely wrote, then write the corrected version.

Explain the performance difference between UNION and UNION ALL, why applying filters outside the union is wasteful, and under what conditions UNION (with deduplication) is actually the right choice.

Skills tested: UNION vs UNION ALL · predicate pushdown · deduplication cost · set operations

5 Build the pre-aggregated summary table

The report from Q1 runs every morning. The team agrees to pre-aggregate results nightly into a summary table so the morning report queries that instead of scanning raw data. You are building it.

Write the DDL for a table called monthly_category_revenue that stores: month, product_category_name, total_revenue, total_orders, and avg_freight.

Then write the DML to populate it — the job must be safe to re-run without creating duplicates (i.e. fully idempotent).

Add at least one index, justify every column type choice, and explain what makes your DML idempotent.

Skills tested: DDL · DML · UPSERT / MERGE · idempotency · index design · data types

SCORING GUIDE

Correctness	Does the SQL produce the right result?
Optimization instinct	Do they identify the bottleneck before rewriting?
Trade-off awareness	Do they acknowledge what their fix costs (storage, complexity, maintenance)?
Scalability thinking	Do they reason about what breaks at 10x data volume?